Bajaj Finserv — Data Science Intern Interview Prep

Complete Question Bank with Answers

SECTION 1: PYTHON & DATA MANIPULATION

Q1. What are the key differences between a list, tuple, and set in Python?

List is mutable, ordered, allows duplicates — `[1, 2, 2, 3]`. **Tuple** is immutable, ordered, allows duplicates — `(1, 2, 2, 3)`. **Set** is mutable, unordered, no duplicates — `{1, 2, 3}`. Use lists when you need ordered mutable collections, tuples for fixed data (like returning multiple values from a function or dictionary keys), and sets for membership testing or deduplication since lookup is O(1).

Q2. What is the difference between `deepcopy` and shallow copy?

Shallow copy (`copy.copy()` or `list.copy()`) creates a new outer object but references the same nested objects. Deep copy (`copy.deepcopy()`) recursively copies all nested objects. Example: if you shallow-copy a list of lists and modify an inner list, the change reflects in both copies. Deep copy prevents this.

Q3. What are list comprehensions and generator expressions? When would you use each?

List comprehension `[x**2 for x in range(1000)]` creates the entire list in memory upfront. Generator expression `(x**2 for x in range(1000))` yields one item at a time lazily. Use generators when working with large datasets where you don't need all values simultaneously — saves memory. Use list comprehensions when you need random access or the full collection.

Q4. Explain `*args` and `**kwargs`.

`*args` collects positional arguments into a tuple: `def f(*args)` allows `f(1, 2, 3)` where args = (1, 2, 3). `**kwargs` collects keyword arguments into a dictionary: `def f(**kwargs)` allows `f(a=1, b=2)` where kwargs = {'a': 1, 'b': 2}. Commonly used in wrapper functions, decorators, and when forwarding arguments.

Q5. What is the difference between `is` and `==`?

`==` checks value equality (calls `__eq__`). `is` checks identity — whether two variables point to the same object in memory. `a = [1, 2]; b = [1, 2]` → `a == b` is True but `a is b` is False. Use `is` mainly for `None` checks: `if x is None`.

Q6. Explain decorators in Python with an example.

A decorator is a function that takes another function as input, adds behavior, and returns a modified function. Common use: timing, logging, authentication.

```python
import time
def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time() - start:.2f}s")
        return result
    return wrapper

@timer
def train_model(data):
    # training logic
    pass
```

Q7. What does `pandas.DataFrame.groupby()` do? How is it different from `pivot_table()`?

`groupby()` splits the DataFrame by one or more columns, applies an aggregation function, and combines results. `pivot_table()` is a higher-level reshaping tool that creates a spreadsheet-style table with custom rows, columns, and aggregation — essentially a `groupby` + `unstack` in one step.

```python
# groupby — flat output
df.groupby('district')['response_time'].mean()

# pivot_table — matrix output
df.pivot_table(values='response_time', index='district', columns='shift', aggfunc='m
```

Q8. How do you handle missing values in pandas?

First, diagnose with `df.isnull().sum()` and `df.isnull().mean()` (percentage). Then decide:

- **Drop** rows/columns: `df.dropna()` — when missing % is very small (<5%) and data is MCAR (Missing Completely At Random).
- **Impute** with mean/median: `df.fillna(df.median())` — for numerical columns when data is roughly symmetric (use median) or normal (use mean).
- **Forward/backward fill:** `df.fillna(method='ffill')` — for time-series data where previous values are reasonable proxies.
- **Model-based imputation**: Using KNNImputer or IterativeImputer from scikit-learn when relationships between features can predict missing values.
- **Flag missingness**: Create a binary indicator column `df['col_missing'] = df['col'].isnull().astype(int)` — when the fact that data is missing is itself

informative.

Q9. Explain the difference between `apply()`, `map()`, and `applymap()`.

- `map()` works on a **Series** only, element-wise. Used for substitution or element-level transforms: `df['col'].map(lambda x: x*2)`.
- `apply()` works on both **Series and DataFrame**. On a DataFrame, it applies a function along an axis (row or column): `df.apply(np.sum, axis=0)`.
- `applymap()` works on a **DataFrame** only, element-wise (deprecated in newer pandas — use `map()` on DataFrame instead): `df.applymap(lambda x: round(x, 2))`.

Q10. How would you merge/join two DataFrames? Explain different join types.

```python
pd.merge(df1, df2, on='key', how='inner')  # only matching rows
pd.merge(df1, df2, on='key', how='left')   # all rows from df1, matching from df2
pd.merge(df1, df2, on='key', how='right')  # all rows from df2, matching from df1
pd.merge(df1, df2, on='key', how='outer')  # all rows from both, NaN where no match
```

`pd.concat()` stacks DataFrames vertically (axis=0) or horizontally (axis=1) — doesn't require a key column.

Q11. How do you optimize pandas for large datasets (millions of rows)?

- Use `dtype` optimization: convert `object` columns to `category` for low-cardinality strings, downcast numerics (`int64` → `int32`).
- Read in chunks: `pd.read_csv('file.csv', chunksize=100000)`.
- Use `query()` instead of boolean indexing for readability and sometimes performance.
- Avoid iterating rows — vectorize operations.
- Use `pyarrow` or `modin` as backend engines.
- For IIT Madras 3.59M records: I used chunked reads, category dtypes for district/emergency_type columns, and processed aggregations per chunk before combining.

Q12. What is the difference between `loc` and `iloc`?

`loc` is **label-based** — `df.loc[0:5, 'column_name']` includes row with label 5. `iloc` is **integer position-based** — `df.iloc[0:5, 0]` excludes position 5. Key gotcha: if your index is non-sequential integers (e.g., after filtering), `loc[5]` gives the row labeled 5, `iloc[5]` gives the 6th row.

SECTION 2: SQL

Q13. Write a query to find the second-highest salary from an Employee table.

```sql
-- Method 1: Using LIMIT/OFFSET
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;

-- Method 2: Using subquery
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);

-- Method 3: Using DENSE_RANK (most flexible for Nth highest)
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) ranked
WHERE rnk = 2;
```

Q14. Explain different types of JOINs with examples.

- **INNER JOIN**: Returns rows with matching keys in both tables. Use: finding customers who have placed orders.
- **LEFT JOIN**: All rows from left table + matching from right (NULLs for no match). Use: all customers including those with no orders.
- **RIGHT JOIN**: Mirror of LEFT JOIN.
- **FULL OUTER JOIN**: All rows from both tables. Use: reconciliation — finding mismatches between two datasets.
- **CROSS JOIN**: Cartesian product — every row in table A paired with every row in table B. Use: generating all possible combinations (e.g., all product-store pairs).
- **SELF JOIN**: Joining a table with itself. Use: finding employees who share the same manager.

Q15. What are window functions? Give examples.

Window functions perform calculations across a set of rows related to the current row without collapsing them (unlike GROUP BY).

```sql
```

```sql
-- Ranking
SELECT name, department, salary,
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank
FROM employees;

-- Running total
SELECT date, revenue,
    SUM(revenue) OVER (ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT RO
FROM daily_sales;

-- Previous value comparison
SELECT month, revenue,
    LAG(revenue, 1) OVER (ORDER BY month) AS prev_month,
    revenue - LAG(revenue, 1) OVER (ORDER BY month) AS month_over_month
FROM monthly_sales;
```

Key functions: `ROW_NUMBER()` (unique sequential), `RANK()` (ties get same rank, skips next), `DENSE_RANK()` (ties get same rank, no skip), `LAG()`/`LEAD()` (previous/next row), `NTILE()` (bucketing).

Q16. What is the difference between WHERE and HAVING?

`WHERE` filters rows **before** grouping. `HAVING` filters groups **after** aggregation.

sql

```sql
-- WHERE: filter individual transactions > 1000
SELECT department, AVG(amount) FROM transactions
WHERE amount > 1000
GROUP BY department;

-- HAVING: filter departments where average > 1000
SELECT department, AVG(amount) FROM transactions
GROUP BY department
HAVING AVG(amount) > 1000;
```

Q17. Explain CTEs vs Subqueries. When would you prefer one over the other?

A CTE (Common Table Expression) is a named temporary result set defined with `WITH`:

sql

```sql
WITH high_value_customers AS (
    SELECT customer_id, SUM(amount) as total
    FROM orders GROUP BY customer_id HAVING SUM(amount) > 10000
)
SELECT c.name, h.total FROM customers c JOIN high_value_customers h ON c.id = h.cust
```

CTEs are preferred when: the logic is reused multiple times in the same query, readability matters, or you need recursion. Subqueries are fine for simple, one-off nested conditions. Performance-wise, most modern databases optimize them identically.

Q18. What are indexes and when would you use them?

An index is a data structure (usually B-tree) that speeds up data retrieval at the cost of slower writes and extra storage. Use indexes on: columns frequently used in WHERE, JOIN, and ORDER BY clauses. Avoid on: columns with low cardinality (e.g., boolean), tables with heavy INSERT/UPDATE operations, or very small tables. Composite indexes cover multiple columns — column order matters (leftmost prefix rule).

Q19. What is normalization? Explain 1NF, 2NF, 3NF.

Normalization reduces data redundancy and improves integrity.

- **1NF**: Each cell contains atomic (single) values; each row is unique.
- **2NF**: 1NF + no partial dependencies (every non-key column depends on the entire primary key, not just part of it).
- **3NF**: 2NF + no transitive dependencies (non-key columns don't depend on other non-key columns).

Example violation of 3NF: storing both `zip_code` and `city` — city depends on zip_code, not on the primary key directly.

Q20. Difference between SQL and NoSQL databases?

SQL (PostgreSQL, MySQL): Structured schemas, ACID transactions, relational model, best for complex queries and data integrity. **NoSQL**: flexible schemas — Document stores (MongoDB: JSON docs, good for nested/variable structures), Key-value (Redis: caching, sessions), Column-family (Cassandra: high write throughput, time-series), Graph (Neo4j: relationships). Choose NoSQL when: schema changes frequently, data is unstructured, horizontal scaling is critical, or access patterns are simple key lookups.

SECTION 3: STATISTICS & HYPOTHESIS TESTING

Q21. What is a p-value? How do you interpret it?

The p-value is the probability of observing data as extreme as (or more extreme than) what was actually observed, **assuming the null hypothesis is true**. If p-value < significance level (commonly 0.05), we reject the null hypothesis. Important: a low p-value does NOT prove the alternative hypothesis is true — it says the data is unlikely under the null. A p-value of 0.03 means there's a 3% chance of seeing this result if the null were true.

Q22. Explain Type I and Type II errors.

- **Type I (False Positive)**: Rejecting the null hypothesis when it's actually true. Example: concluding a new loan approval model is better when it's actually not. Probability = α (significance level).
- **Type II (False Negative)**: Failing to reject the null when it's actually false. Example: concluding there's no difference when the new model actually is better. Probability = β. Power = 1 - β.

Trade-off: reducing α increases β and vice versa. In fraud detection, you'd want low Type II (don't miss actual fraud) even at the cost of higher Type I (some false alarms).

Q23. How would you design an A/B test for a new feature?

1. **Define hypothesis**: H_0 = new feature has no effect on conversion. H_1 = it does.
2. **Choose metric**: Primary KPI (e.g., conversion rate) and guardrail metrics (e.g., page load time shouldn't degrade).
3. **Calculate sample size**: Using power analysis — need significance level (α=0.05), power (0.8), and minimum detectable effect (e.g., 2% lift).
4. **Randomize**: Randomly assign users to control (A) and treatment (B). Ensure no selection bias.
5. **Run experiment**: For a pre-determined duration — don't peek and stop early (peeking inflates false positives).
6. **Analyze**: Use appropriate test (chi-square for proportions, t-test for means). Check for statistical and **practical** significance.
7. **Watch for pitfalls**: Simpson's paradox (results flip when segmented), novelty effect, network effects.

Q24. Explain confidence intervals.

A 95% confidence interval means: if we repeated this experiment 100 times, approximately 95 of the resulting intervals would contain the true population parameter. It is NOT "95% probability the true value is in this range." Width depends on: sample size (larger → narrower), variability in data, and confidence level chosen.

Q25. When do you use t-test vs z-test vs chi-square test?

- **Z-test**: Comparing means when population variance is known AND sample size is large (n > 30). Rarely used in practice.
- **T-test**: Comparing means when population variance is unknown (almost always). One-sample (vs known value), two-sample (independent groups), paired (before-after).
- **Chi-square test**: For categorical data — testing independence between two categorical variables or goodness of fit. Example: is loan default rate independent of the customer's

city?

Q26. What is the Central Limit Theorem and why does it matter?

The CLT states that the sampling distribution of the sample mean approaches a normal distribution as sample size increases, regardless of the population's distribution — provided the samples are independent and identically distributed. It matters because it justifies using normal-distribution-based tests (z-test, t-test) even when the underlying data isn't normal, as long as sample size is large enough (typically $n \geqslant 30$).

Q27. Explain Bayes' Theorem with a practical example.

$P(A|B) = P(B|A) \times P(A) / P(B)$

Example: A fraud detection system flags 90% of actual fraud (sensitivity) and has a 5% false positive rate. If 1% of transactions are truly fraudulent, what's the probability a flagged transaction is actually fraud?

$P(\text{Fraud}|\text{Flagged}) = P(\text{Flagged}|\text{Fraud}) \times P(\text{Fraud}) / P(\text{Flagged}) = (0.90 \times 0.01) / (0.90 \times 0.01 + 0.05 \times 0.99) = 0.009 / 0.0585 \approx \mathbf{15.4\%}$

Even with a 90% detection rate, only ~15% of flagged transactions are actual fraud — because fraud is rare (base rate problem).

Q28. What are common probability distributions and their use cases?

- **Normal**: Continuous, bell-shaped. Heights, measurement errors, stock returns (approximately).
- **Binomial**: Discrete, n trials with probability p. Loan default (yes/no), click-through events.
- **Poisson**: Discrete, count of events in fixed interval. Number of claims per month, server requests per second.
- **Exponential**: Continuous, time between Poisson events. Time between customer arrivals.
- **Uniform**: Equal probability across range. Random number generation, initial weight initialization.

Q29. What is correlation vs causation?

Correlation measures the linear relationship between two variables (-1 to +1). Causation means one variable directly influences the other. Ice cream sales and drowning deaths are correlated (both increase in summer) but ice cream doesn't cause drowning — temperature is the confounding variable. Establishing causation requires: controlled experiments (A/B tests), temporal precedence, ruling out confounders, or quasi-experimental methods (difference-in-differences, instrumental variables).

SECTION 4: MACHINE LEARNING

Q30. Explain the bias-variance tradeoff.

Bias = error from wrong assumptions in the model (underfitting). High bias means the model is too simple. **Variance** = error from sensitivity to small fluctuations in training data (overfitting). High variance means the model is too complex. **Total error = Bias² + Variance + Irreducible noise**. The sweet spot minimizes both — achieved through proper model complexity, regularization, and cross-validation.

Q31. How do you handle overfitting?

- **More training data** (best solution when possible).
- **Regularization**: L1 (Lasso — drives weights to zero, feature selection) or L2 (Ridge — shrinks weights, keeps all features).
- **Cross-validation**: k-fold CV to get robust performance estimates.
- **Early stopping**: Stop training when validation loss starts increasing.
- **Dropout** (for neural networks): Randomly zero out neurons during training.
- **Feature selection**: Remove noisy or irrelevant features.
- **Ensemble methods**: Bagging (Random Forest) reduces variance.

Q32. Explain L1 (Lasso) vs L2 (Ridge) regularization.

Both add a penalty to the loss function to prevent overfitting.

- **L1 (Lasso)**: Adds |w| penalty. Drives some weights to exactly zero → automatic feature selection. Produces sparse models. Use when you suspect many features are irrelevant.
- **L2 (Ridge)**: Adds w^2 penalty. Shrinks all weights toward zero but never exactly zero. Handles multicollinearity well. Use when most features are useful.
- **Elastic Net**: Combines both L1 and L2. Use when you want feature selection but also have correlated features.

Q33. Walk through building a classification model end-to-end.

1. **Problem framing**: Define target variable, success metric, business context.
2. **Data collection & exploration**: EDA — distributions, correlations, class balance, missing values.
3. **Preprocessing**: Handle missing values, encode categoricals, scale features.
4. **Feature engineering**: Create domain-specific features, interaction terms, time-based features.
5. **Train-test split**: Stratified split (70-30 or 80-20). Never touch test set during development.

6. **Model selection**: Start simple (Logistic Regression) → tree-based (Random Forest, XGBoost) → compare.
7. **Hyperparameter tuning**: GridSearchCV or RandomizedSearchCV with cross-validation.
8. **Evaluation**: Accuracy, Precision, Recall, F1, AUC-ROC depending on business need.
9. **Interpretation**: Feature importance, SHAP values, partial dependence plots.
10. **Deployment & monitoring**: Model serving, drift detection, retraining pipeline.

Q34. Explain precision, recall, F1-score, and AUC-ROC.

- **Precision**: TP / (TP + FP). Of all predicted positives, how many are correct? Optimize when false positives are costly (spam filter — don't send legit email to spam).
- **Recall (Sensitivity)**: TP / (TP + FN). Of all actual positives, how many were detected? Optimize when false negatives are costly (cancer detection — don't miss actual cancer).
- **F1-Score**: Harmonic mean of precision and recall. 2×(P×R)/(P+R). Use when you need a single balanced metric.
- **AUC-ROC**: Area under the Receiver Operating Characteristic curve (True Positive Rate vs False Positive Rate at various thresholds). 0.5 = random, 1.0 = perfect. Good for comparing models across all thresholds.

Q35. What is the difference between bagging and boosting?

- **Bagging** (Bootstrap Aggregating): Train multiple models on random subsets of data in **parallel**. Average predictions (regression) or majority vote (classification). Reduces **variance**. Example: Random Forest.
- **Boosting**: Train models **sequentially** — each model focuses on errors of the previous one. Reduces **bias**. Examples: AdaBoost, Gradient Boosting, XGBoost, LightGBM.
- Key difference: Bagging reduces variance (prevents overfitting), Boosting reduces bias (increases model power). Boosting is generally more powerful but more prone to overfitting if not tuned.

Q36. How does XGBoost work? Why did you choose it for the IIT Madras project?

XGBoost (Extreme Gradient Boosting) builds decision trees sequentially. Each new tree fits the **gradient of the loss** (residual errors) from the previous ensemble. Key features: L1/L2 regularization on tree weights, column subsampling (like Random Forest), handling missing values natively, and parallel tree construction at the split-finding level.

Why XGBoost at IIT Madras: We needed to predict expected ambulance response times (regression task) with interpretability. XGBoost handled the mix of numerical and categorical features (district, emergency type, time of day) well, its regularization prevented overfitting on the 3.59M records, and feature importance was straightforward to extract. We validated with

WMAPE (8.37%) because it weights errors by magnitude — a 5-minute error on a 10-minute response is worse than on a 60-minute response.

Q37. Explain how Random Forest works.

Random Forest builds N decision trees, each trained on a bootstrapped sample (random subset with replacement) of the training data. At each split, only a random subset of features is considered (√p for classification, p/3 for regression typically). Final prediction is the average (regression) or majority vote (classification) of all trees. This double randomness (data + features) decorrelates the trees, reducing variance significantly compared to a single deep tree.

Q38. What is cross-validation and why is it important?

Cross-validation estimates model performance on unseen data. **K-fold CV**: split data into k equal parts, train on k-1 folds, validate on the remaining fold, rotate k times, and average scores. This gives a more robust estimate than a single train-test split. **Stratified k-fold**: preserves class distribution in each fold — crucial for imbalanced datasets. Use CV for: model comparison, hyperparameter tuning, estimating generalization error.

Q39. How do you handle class imbalance?

- **Resampling**: Oversample minority class (SMOTE — creates synthetic samples by interpolating between neighbors) or undersample majority class.
- **Class weights**: Pass `class_weight='balanced'` or custom weights to the model — penalizes misclassifying minority class more heavily.
- **Threshold tuning**: Adjust classification threshold from default 0.5 to optimize for recall.
- **Use appropriate metrics**: Accuracy is misleading. Use precision, recall, F1, or AUC-PR (Precision-Recall curve, better than AUC-ROC for severe imbalance).
- **Anomaly detection approach**: If positive class is very rare (<1%), treat it as anomaly detection (Isolation Forest, Autoencoders).

At IIT Madras: For our autoencoder-based anomaly detection (8,567 outliers from 3.59M records — ~0.24%), we used an unsupervised approach specifically because the "anomalous" class was too rare and not well-defined for supervised learning.

Q40. Explain Logistic Regression. Is it a classification or regression algorithm?

Despite the name, it's a **classification** algorithm. It models the probability of the positive class using the sigmoid function: P(y=1|x) = 1 / (1 + e^(-z)) where z = w·x + b. The sigmoid squashes output to [0, 1]. Decision boundary is typically at 0.5. The loss function is binary cross-entropy (log loss), optimized via gradient descent. Assumptions: linear relationship between features and log-odds, no multicollinearity, observations are independent.

Q41. What is the difference between supervised, unsupervised, and reinforcement learning?

- **Supervised**: Labeled data (input-output pairs). Classification (predict category) and Regression (predict continuous value). Examples: spam detection, house price prediction.

- **Unsupervised**: No labels. Discover patterns — Clustering (K-means, DBSCAN), Dimensionality Reduction (PCA, t-SNE), Anomaly Detection (Isolation Forest, Autoencoders).
- **Reinforcement**: Agent learns by interacting with environment, receiving rewards/penalties. Examples: game playing, robotics, recommendation systems.

Q42. Explain K-Means clustering. What are its limitations?

K-Means partitions data into K clusters by: (1) randomly initialize K centroids, (2) assign each point to nearest centroid, (3) recompute centroids as mean of assigned points, (4) repeat until convergence. Limitations: must specify K upfront (use elbow method or silhouette score), assumes spherical clusters of similar size, sensitive to initialization (use K-Means++), sensitive to outliers, only finds convex clusters (use DBSCAN for arbitrary shapes).

Q43. What is PCA and when do you use it?

Principal Component Analysis finds orthogonal directions (principal components) of maximum variance in the data. The first PC captures the most variance, the second captures the next most (perpendicular to the first), and so on. Use PCA for: dimensionality reduction (reduce 100 features to 10 while retaining 95% variance), visualization (project to 2D/3D), removing multicollinearity, speeding up training. Limitation: components are linear combinations — hard to interpret. For non-linear reduction, use t-SNE or UMAP.

Q44. What is gradient descent? Explain its variants.

Gradient descent minimizes a loss function by iteratively moving in the direction of the steepest descent (negative gradient).

- **Batch GD**: Computes gradient over entire dataset. Stable but slow for large data.
- **Stochastic GD (SGD)**: Computes gradient for one sample. Fast, noisy, can escape local minima.
- **Mini-batch GD**: Computes gradient over a small batch (32, 64, 128). Best of both — stable and efficient. Most commonly used.

Learning rate is critical: too high → diverge, too low → slow convergence. Solutions: learning rate schedulers, Adam optimizer (adaptive learning rate per parameter).

SECTION 5: FEATURE ENGINEERING

Q45. How do you handle categorical variables?

- **One-Hot Encoding**: Creates binary columns for each category. Good for nominal categories with few unique values. `pd.get_dummies()`. Problem: high cardinality → curse of dimensionality.

- **Label Encoding**: Maps categories to integers (0, 1, 2...). Good for ordinal data (low/medium/high). Don't use for nominal data — model assumes ordering.
- **Target Encoding**: Replace category with mean of target variable for that category. Powerful but risks data leakage — always compute on training fold only.
- **Frequency Encoding**: Replace with count/frequency of that category. Simple and effective.
- **Binary/Hashing**: For very high cardinality (e.g., zip codes with 10000+ values).

Q46. When and why do you scale features?
Feature scaling is necessary when: algorithms use distance metrics (KNN, SVM, K-Means), algorithms use gradient descent (Neural Networks, Logistic Regression), or features have very different magnitudes. **Not needed** for tree-based models (Random Forest, XGBoost) — they split on thresholds, not magnitudes.

- **StandardScaler**: $z = (x - \mu) / \sigma$. Mean=0, std=1. Use when data is roughly Gaussian.
- **MinMaxScaler**: $x' = (x - min) / (max - min)$. Scales to [0, 1]. Use when you need bounded values. Sensitive to outliers.
- **RobustScaler**: Uses median and IQR. Robust to outliers.

Q47. What is multicollinearity and how do you detect/handle it?

Multicollinearity: two or more features are highly correlated, making it hard to isolate individual effects. Detection: correlation matrix ($|r| > 0.8$), VIF (Variance Inflation Factor) > 5 or 10 is concerning. Effects: inflated standard errors, unstable coefficients, unreliable feature importance. Handling: remove one of the correlated features, use PCA, use regularization (Ridge handles it well), or use tree-based models (less affected).

Q48. Explain feature selection methods.

- **Filter methods**: Statistical tests independent of model. Correlation with target, chi-square test, mutual information. Fast but ignores feature interactions.
- **Wrapper methods**: Train model with different feature subsets. Forward selection, backward elimination, recursive feature elimination (RFE). Accurate but computationally expensive.
- **Embedded methods**: Feature selection built into model training. Lasso (L1), tree-based feature importance, Elastic Net. Good balance of accuracy and speed.

SECTION 6: DEEP LEARNING

Q49. What is a neural network? Explain forward and backward propagation.

A neural network is layers of interconnected nodes (neurons). Each connection has a weight. **Forward propagation**: input flows through layers, each neuron computes weighted sum + bias →

activation function → output. **Backward propagation**: compute loss at output → compute gradient of loss w.r.t. each weight using chain rule → update weights using gradient descent. This iterates until loss converges.

Q50. Explain CNNs. Why are they good for images?

CNNs use convolutional layers that apply learnable filters (kernels) across the input image. Key intuition: local patterns (edges, textures) are detected by small filters that share weights across the image — this provides **translation invariance** (a cat is a cat regardless of position) and dramatically reduces parameters compared to fully connected layers. Architecture: Conv layers (extract features) → Pooling layers (downsample, reduce computation) → Fully connected layers (classify). In my U-Net project, the encoder path uses convolutions to extract denoising features, while the decoder path uses upsampling + skip connections to reconstruct the clean image.

Q51. What is the difference between RNN, LSTM, and Transformers?

- **RNN**: Processes sequential data with hidden state. Problem: vanishing/exploding gradients on long sequences.
- **LSTM**: Solves vanishing gradient with gates (forget, input, output) that control information flow. Can capture long-term dependencies. In my AutoScaler project, I use LSTM autoencoders for detecting slow performance degradation patterns.
- **Transformers**: Use self-attention to weigh all positions simultaneously (no sequential processing). Parallelizable, captures global context. Foundation of BERT, GPT. Dominant for NLP and increasingly for vision (ViT).

Q52. What is transfer learning?

Using a pre-trained model (trained on a large dataset like ImageNet) and adapting it to a new task. Two approaches: (1) **Feature extraction**: freeze pre-trained layers, add new classification head. (2) **Fine-tuning**: unfreeze some/all layers and train with a small learning rate. Benefits: works with limited data, faster training, better performance. Example: using ResNet pre-trained on ImageNet for medical image classification by fine-tuning the last few layers.

Q53. Explain your U-Net architecture for CT denoising.

U-Net has an encoder-decoder structure with skip connections. The **encoder** (downsampling path) extracts hierarchical features through conv + pooling layers — capturing what noise looks like. The **decoder** (upsampling path) reconstructs the clean image using transposed convolutions. **Skip connections** concatenate encoder features with decoder features at each level — this preserves spatial details that get lost during downsampling, which is critical for medical imaging where fine structures matter. I used mixed precision training (FP16 where safe, FP32 for critical operations) for ~4× faster convergence and a custom data generator that dynamically pairs augmented noisy/clean images to avoid loading all 300+ scans into RAM.

Q54. What are activation functions and when do you use each?

- **ReLU**: $f(x) = \max(0, x)$. Default for hidden layers. Fast, avoids vanishing gradient. Problem:

"dying ReLU" — neurons outputting 0 permanently.

- **Leaky ReLU**: f(x) = x if x>0, αx otherwise. Fixes dying ReLU.
- **Sigmoid**: f(x) = 1/(1+e^-x). Output [0,1]. Used in binary classification output layer.
- **Softmax**: Generalizes sigmoid to multi-class. Outputs sum to 1 (probability distribution). Used in multi-class output layer.
- **Tanh**: f(x) = (e^x - e^-x)/(e^x + e^-x). Output [-1,1]. Sometimes used in RNNs/LSTMs.

SECTION 7: DATA VISUALIZATION & DASHBOARDS

Q55. What makes a good dashboard?

A good dashboard answers a specific question for a specific audience. Key principles: (1) **Clear hierarchy** — most important KPI is largest and top-left, (2) **Appropriate chart types** — don't use pie charts for >5 categories, use line charts for trends over time, (3) **Minimal clutter** — remove chart junk, unnecessary gridlines, 3D effects, (4) **Consistent formatting** — same colors for same categories across charts, (5) **Interactivity** — filters and drill-downs for exploration, (6) **Context** — show targets, benchmarks, or prior period comparisons.

Q56. When would you use different chart types?

- **Bar chart**: Comparing categorical values (revenue by product).
- **Line chart**: Trends over time (daily active users).
- **Scatter plot**: Relationship between two continuous variables (age vs income).
- **Heatmap**: Matrix relationships (correlation matrix, confusion matrix).
- **Histogram**: Distribution of a single variable.
- **Box plot**: Distribution summary + outliers across categories.
- **Area chart**: Stacked trends showing part-to-whole over time.

Q57. Describe your Power BI experience at Hindalco.

At Hindalco, I built dashboards tracking sensor readings from the manufacturing floor — temperature, vibration, pressure across multiple assets. The dashboards integrated with Proficy Historian data and showed: real-time anomaly flags from our ARIMA models, historical trend lines with confidence bands, asset health scores with color-coded status indicators, and maintenance schedule views. The key metric was unscheduled downtime — we tracked it pre- and post-model deployment and demonstrated the 20% reduction.

SECTION 8: WEB SCRAPING & APIs

Q58. What Python libraries would you use for web scraping?

- **BeautifulSoup** + requests: For simple static HTML pages. Parse HTML, find elements by tag/class/id.
- **Scrapy**: Full framework for large-scale scraping. Handles pagination, rate limiting, data pipelines out of the box.
- **Selenium** / Playwright: For JavaScript-rendered pages (SPAs). Controls a real browser. Slower but necessary when content loads dynamically.
- **Pandas** `pd.read_html()`: Quick extraction of HTML tables.

Always respect `robots.txt`, use delays between requests, and check terms of service.

Q59. Explain REST APIs. What are HTTP methods and status codes?

REST (Representational State Transfer) is an architectural style for web APIs using HTTP. **Methods**: GET (read), POST (create), PUT (update/replace), PATCH (partial update), DELETE (remove). **Status codes**: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests (rate limited), 500 Internal Server Error.

Handling API pagination: follow `next_page` links or use offset/cursor parameters. Rate limiting: implement exponential backoff and respect `Retry-After` headers.

Q60. What is the difference between REST and GraphQL?

REST: Multiple endpoints (`/users`, `/users/1/orders`), server decides response shape, over-fetching or under-fetching common. **GraphQL**: Single endpoint, client specifies exactly what data it needs in the query, returns only requested fields. GraphQL is better when: clients need flexible data fetching (mobile vs desktop), relationships are complex, or you want to minimize network requests. REST is simpler for straightforward CRUD operations.

SECTION 9: RESUME DEEP-DIVES

Q61. Walk us through your IIT Madras autoencoder anomaly detection pipeline.

Phase I used an **unsupervised autoencoder** — a neural network that learns to compress and reconstruct normal dispatch data. It was trained only on normal records. At inference time, records with high reconstruction error (the model couldn't accurately rebuild them) were flagged as anomalous. We identified 8,567 outliers out of 3.59M records. The threshold was set using the distribution of reconstruction errors — points beyond the 99.7th percentile were flagged.

Why unsupervised? Because we had no labeled "anomalous" dispatches — we didn't know a priori what constituted an unusual dispatch. The autoencoder learned the manifold of normal

behavior and flagged deviations.

From these anomalies, I constructed a **Time Drift Index (TDI)** — measuring how much actual response times drifted from expected baselines over time for each district.

Q62. What is the Quantum Stability Index (QSI) and how did you design it?

QSI is a fairness-aware composite metric I designed to classify district-level operational performance. It normalizes the Time Drift Index against call volume — because a district handling 500 calls/day with 10% drift is operationally different from one handling 50 calls/day with the same drift. The formula weights: (1) magnitude of time drift, (2) consistency/variance of drift, (3) call volume as a normalizing factor. Districts were classified into 5 stability bands (from "Highly Stable" to "Critical") using the composite score, enabling policymakers to prioritize resource allocation.

Q63. Explain ARIMA from your Hindalco work. How did you determine parameters?

ARIMA(p, d, q): **p** = autoregressive order (how many past values), **d** = differencing order (how many times to difference for stationarity), **q** = moving average order (how many past forecast errors).

Steps:

1. **Stationarity check**: ADF test (Augmented Dickey-Fuller). If p-value > 0.05, data is non-stationary → difference it (d=1 or d=2).
2. **Determine p and q**: ACF plot (autocorrelation function) — significant spikes suggest q value. PACF plot (partial autocorrelation) — significant spikes suggest p value.
3. **Alternative**: Use `auto_arima` from pmdarima which searches over (p, d, q) combinations and selects by AIC/BIC.
4. **Validation**: Checked residuals for white noise (no autocorrelation), used walk-forward validation on time-series (can't use random k-fold — temporal ordering matters).

Q64. How does your AutoScaler's 3-tier detection system work?

Layer 1 — Isolation Forest: Detects multivariate spike anomalies. It isolates observations by randomly selecting a feature and split value. Anomalies require fewer splits to isolate (shorter path length in the tree). Good for sudden, dramatic metric changes.

Layer 2 — LSTM Autoencoder: Detects slow degradation patterns. Trained on time-series windows of normal metrics. When reconstruction error exceeds threshold over a sustained period, it flags gradual drift that point-in-time methods miss.

Layer 3 — Claude API (LLM reasoning): Takes detected anomalies, their SHAP feature attributions, and system context → produces natural language root cause analysis and suggested remediation. If the API is unavailable, a Random Forest classifier trained on historical incident data activates as fallback, returning the same JSON response schema.

The 7-day cold-start warm-up prevents false alerts during initial deployment when the models don't have enough baseline data.

SECTION 10: BAJAJ FINSERV BUSINESS SCENARIOS

Q65. How would you build a credit scoring model?

Data: Customer demographics, income, employment history, existing loan history, repayment patterns, credit bureau data, transaction behavior. **Features**: debt-to-income ratio, number of missed payments in last 12 months, credit utilization %, length of credit history, number of recent hard inquiries, account-level aggregations from transaction data. **Model**: Logistic Regression (interpretable, regulatory compliance) or XGBoost (better performance). **Metric**: AUC-ROC for discrimination, KS statistic for rank-ordering. Must ensure **fairness** — check disparate impact across protected groups. Use SHAP for explainability — regulators require reasons for denial.

Q66. How would you detect fraudulent transactions?

Approach: Anomaly detection + supervised classification hybrid. **Features**: transaction amount deviation from user's mean, time since last transaction, geographic distance from last transaction, merchant category frequency, device fingerprint changes, velocity checks (transactions per hour). **Model**: Isolation Forest for unsupervised detection (catches novel fraud patterns) + XGBoost/LightGBM for known fraud patterns. **Challenges**: extreme class imbalance (fraud < 0.1%), concept drift (fraud patterns evolve), latency requirements (real-time scoring), minimizing false positives (declining legitimate transactions hurts customer trust). **Evaluation**: Precision-Recall curve (not ROC — due to imbalance), and optimize for recall at acceptable precision threshold set by business.

Q67. How would you build a recommendation engine for financial products?

Collaborative filtering: Users who took a personal loan often also take credit cards → recommend credit cards to personal loan holders. **Content-based**: Match product features (interest rate, tenure, benefits) with user preferences derived from browsing/interaction history. **Hybrid approach**: Combine both with a two-stage system — candidate generation (retrieve top 100 relevant products) → ranking (score and sort by predicted engagement/conversion). **Features**: user's age, income bracket, existing products, life events (marriage, new job), browsing patterns, seasonal trends (tax-saving investments in Jan-Mar).

Q68. How would you design an A/B test for a new loan approval algorithm?

Setup: Randomly assign incoming loan applications to two cohorts — Control (existing algorithm) and Treatment (new algorithm). **Primary metric**: Approval-to-default ratio (want higher approvals without more defaults). **Guardrail metrics**: Overall approval rate (don't want it to drop dramatically), time-to-decision, customer satisfaction. **Duration**: Need enough time for defaults to materialize (30-90 days minimum for short-term loans). **Risk mitigation**: Start with a small treatment group (10%) and scale up. Monitor continuously for any segment getting

disproportionately affected. **Analysis**: Compare default rates between cohorts using survival analysis (since defaults happen over time, not all at once).

Q69. How would you analyze customer churn for Bajaj Finserv?

Define churn: No transactions in 90 days, account closure, or policy lapse — depends on product. **Features**: recency of last transaction, frequency and monetary value (RFM), tenure, number of products held, complaint history, payment delays, engagement with app/website, life events. **Model**: Survival models (Cox regression) for time-to-churn, XGBoost for binary churn prediction, segment using clustering to identify distinct churn profiles. **Actionable output**: Score customers daily, trigger retention offers for high-risk segments, identify which intervention works for which segment.

SECTION 11: CODING CHALLENGES (LIKELY IN INTERVIEW)

Q70. Write a function to compute moving averages.

python

```python
def moving_average(data, window_size):
    """Compute simple moving average."""
    result = []
    for i in range(len(data) - window_size + 1):
        window = data[i:i + window_size]
        result.append(sum(window) / window_size)
    return result

# Using pandas (preferred in practice):
import pandas as pd
df['ma_7'] = df['value'].rolling(window=7).mean()
```

Q71. Implement K-Means from scratch.

python

```python
import numpy as np

def kmeans(X, k, max_iters=100):
    # Random initialization
    indices = np.random.choice(len(X), k, replace=False)
    centroids = X[indices]

    for _ in range(max_iters):
        # Assign clusters
        distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
        labels = np.argmin(distances, axis=1)

        # Update centroids
        new_centroids = np.array([X[labels == i].mean(axis=0) for i in range(k)])

        # Check convergence
        if np.allclose(centroids, new_centroids):
            break
        centroids = new_centroids

    return labels, centroids
```

Q72. Write a function to detect outliers using IQR method.

```python
def detect_outliers_iqr(series):
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = series[(series < lower_bound) | (series > upper_bound)]
    return outliers, lower_bound, upper_bound
```

Q73. Write SQL to find customers who made purchases in every month of 2025.

```sql
SELECT customer_id
FROM purchases
WHERE YEAR(purchase_date) = 2025
GROUP BY customer_id
HAVING COUNT(DISTINCT MONTH(purchase_date)) = 12;
```

Q74. Write a function to calculate WMAPE (Weighted Mean Absolute Percentage Error).

SECTION 12: BEHAVIORAL & SITUATIONAL

Q75. Tell me about yourself.

I'm a B.Tech CS (AI/ML specialization) student at SRM Institute of Science and Technology. My strongest experience is a year-long research internship at IIT Madras, where I co-authored a research paper analyzing 3.59 million ambulance dispatch records from Tamil Nadu's 108 EMS system. I built a dual-phase AI framework — unsupervised anomaly detection with autoencoders and counterfactual benchmarking with XGBoost — to surface operational disparities across 38 districts. Before that, I interned at Hindalco where I built predictive maintenance models using ARIMA and anomaly detection that reduced unscheduled downtime by 20%. I'm drawn to this role because it combines statistical modeling, ML, and business problem-solving — which is exactly where my skills and interests align.

Q76. Describe a challenging technical problem you solved.

At IIT Madras, the core challenge was detecting operational anomalies in ambulance response times without labeled data — we didn't know what "anomalous" looked like. I designed a two-phase approach: first, train an autoencoder on all records to learn what "normal" dispatch behavior looks like, then flag high-reconstruction-error records as anomalous. The non-obvious part was defining the threshold — I analyzed the distribution of reconstruction errors and used a statistical cutoff rather than an arbitrary one. The second challenge was making the findings actionable for policymakers: I created the QSI metric that normalizes drift against call volume, producing 5 interpretable stability bands that directly map to resource allocation decisions.

Q77. How do you handle conflicting priorities or tight deadlines?

At IIT Madras, I simultaneously managed the data pipeline (5+ years of dispatch records with 100+ quality checks) and model development. I broke the work into: must-have (data validation, core model), should-have (SHAP analysis, fairness metrics), and nice-to-have (simulation scenarios). I delivered the must-haves first, then layered on complexity. For the Hindalco internship, I had 2 months to go from raw sensor data to deployed models — I used ARIMA first as a quick baseline, then iterated on custom anomaly techniques.

Q78. Why Bajaj Finserv? Why data science?

Bajaj Finserv operates at the intersection of technology and financial services, which means the

data problems are high-stakes and high-impact — a better credit model or fraud detection system directly affects millions of customers. I'm specifically excited about applying ML to business problems where the outcome is measurable (conversion rates, default rates, operational efficiency) rather than purely academic metrics. My experience at IIT Madras showed me the power of data-driven decisions in public policy; I want to apply the same rigor to financial services at scale.